MODUL PRAKTIKUM 5

Big Data Analytics & Use Case (Decision-Oriented System)

(Industrial Style – Multi-Domain Pipeline: Smart Transportation)

Programming Language	: **Python**
Development Environment	: ***Linux Server Environment***
Code Editor	: **VS Code** (*Remote* ***WSL***)
Command Line Interface	: **Bash**
Distributed Data Processing Engine	: **Apache Spark (PySpark)**
Data Storage	: **Parquet Data Lake**
Real-Time Visualization	: **Streamlit**
Durasi	: **150 – 180 menit**

---

1. TUJUAN PRAKTIKUM

Mahasiswa mampu:

1. Mengembangkan sistem **Big Data Analytics berbasis streaming**
2. Membangun **Decision-Oriented System (Insight + Alert)**
3. Mengelola **multi-domain data pipeline (E-Commerce & Transportation)**
4. Mengintegrasikan:
    - *Data ingestion*
    - *Streaming processing*
    - *Analytics layer*
    - *Dashboard*
5. Menghasilkan **insight berbasis data real-time**

2. KONTEKS INDUSTRI

Dalam industri modern, sistem data tidak hanya menyimpan dan menampilkan data, tetapi:

- ✓ Menghasilkan **insight real-time**
- ✓ Memberikan **alert otomatis**
- ✓ Mendukung **pengambilan keputusan cepat**

Contoh implementasi:

Industri	Use Case
Transportasi	Monitoring kemacetan
E-Commerce	Analisis penjualan
Fintech	Fraud detection
Smart City	Mobilitas & traffic

Pada praktikum ini kita tambahkan 1 **domain** setelah sebelumnya kita menggunakan domain 🛒 E-Commerce, yakni domain :

- 🚗 Smart Transportation

3. ARSITEKTUR PIPELINE

```
Data Generator (JSON)
        ↓
Streaming Layer (Spark Structured Streaming)
        ↓
Data Lake (Parquet)
        ↓
Analytics Engine (Python)
        ↓
Alert System
        ↓
Dashboard (Streamlit)
```

3.1 KONSEP YANG DIGUNAKAN

1. **Streaming Processing**

   Data diproses secara real-time menggunakan **Spark Structured Streaming**

2. **Data Lake (Parquet)**

   Data disimpan dalam format kolumnar untuk efisiensi dan analitik cepat

3. **Analytics Layer**

   Mengubah data menjadi:

   - KPI
   - agregasi
   - pola

4. **Decision-Oriented System**

   Sistem tidak hanya menampilkan data, tetapi:

   - insight
   - alert
   - rekomendasi

5. **Modular Architecture (Multi-Domain)**

   Pipeline dipisahkan berdasarkan domain:

   ```
   ecommerce/
   transportation/
   ```

4. LANGKAH-LANGKAH PRAKTIKUM

LANGKAH 1 — Struktur Project (FINAL)

```
BIGDATA-PROJECT
│
├── scripts/
│   └── transportation/
│
├── analytics/
│   ├── __init__.py    ✅
│   └── transportation_analytics.py
│
├── alerts/
│   ├── __init__.py    ✅
│   └── transportation_alert.py
│
├── dashboard/
│   └── dashboard_transportation.py
│
├── data/
│   ├── serving/
│   │   └── transportation/
│   └── checkpoints/ (akan terbentuk sendiri setelah terminal 1, 2, dan 3 dijalankan)
```

👉 File __init__.py WAJIB agar Python mengenali module

LANGKAH 2 — 🚗 SMART TRANSPORTATION PIPELINE

1️⃣ scripts/transportation/trip_generator.py

```python
import json, time, random, os
from datetime import datetime

OUTPUT_PATH = "stream_data/transportation"
os.makedirs(OUTPUT_PATH, exist_ok=True)

locations = ["Jakarta", "Bandung", "Surabaya"]
vehicles = ["Car", "Motorbike", "Taxi"]

i = 1
while True:
    data = {
        "trip_id": f"TRX{i}",
        "vehicle_type": random.choice(vehicles),
        "location": random.choice(locations),
        "distance": random.uniform(1, 20),
        "fare": random.randint(10000, 100000),
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }
```

```
    with open(f"{OUTPUT_PATH}/trip_{i}.json", "w") as f:
        json.dump(data, f)

    print("Generated Trip:", data)
    i += 1
    time.sleep(3)
```

2 **scripts/transportation/streaming_trip_layer.py**

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import *

spark = SparkSession.builder.appName("TransportationStreaming").getOrCreate()

schema = StructType([
    StructField("trip_id", StringType()),
    StructField("vehicle_type", StringType()),
    StructField("location", StringType()),
    StructField("distance", DoubleType()),
    StructField("fare", DoubleType()),
    StructField("timestamp", StringType())
])

df = spark.readStream.schema(schema).json("stream_data/transportation")

df = df.withColumn("timestamp", to_timestamp("timestamp"))

df.writeStream \
    .format("parquet") \
    .option("path", "data/serving/transportation") \
    .option("checkpointLocation", "data/checkpoints/transportation") \
    .start() \
    .awaitTermination()
```

3 **analytics/transportation_analytics.py**

```
import pandas as pd
import os


# ========================
# LOAD DATA
# ========================
def load_data(path):
    if not os.path.exists(path):
        return pd.DataFrame()

    files = [f for f in os.listdir(path) if f.endswith(".parquet")]
```

```python
    if not files:
        return pd.DataFrame()

    df = pd.concat(
        [pd.read_parquet(os.path.join(path, f)) for f in files],
        ignore_index=True
    )

    return df


# ========================
# PREPROCESS
# ========================
def preprocess(df):
    if df.empty:
        return df

    df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
    df = df.dropna(subset=["timestamp"])

    return df


# ========================
# METRICS
# ========================
def compute_metrics(df):
    if df.empty:
        return {
            "total_trips": 0,
            "total_fare": 0,
            "top_location": "-"
        }
    return {
        "total_trips": len(df),
        "total_fare": df["fare"].sum(),
        "top_location": df.groupby("location")["fare"].sum().idxmax()
    }


# ========================
# PEAK HOUR
# ========================
def detect_peak_hour(df):
    if df.empty:
        return None

    df["hour"] = df["timestamp"].dt.hour
    return df.groupby("hour").size().idxmax()
```

```python
# ========================
# VISUALIZATION DATA
# ========================
def fare_per_location(df):
    if df.empty:
        return pd.Series()

    return df.groupby("location")["fare"].sum().sort_values(ascending=False)


def vehicle_distribution(df):
    if df.empty:
        return pd.Series()

    return df.groupby("vehicle_type").size().sort_values(ascending=False)


def mobility_trend(df):
    if df.empty:
        return pd.Series()

    df = df.set_index("timestamp")
    return df["fare"].resample("10s").sum()


# ========================
# ANOMALY DETECTION
# ========================
def detect_anomaly(df):
    if df.empty:
        return pd.DataFrame()

    # contoh: fare tinggi dianggap anomali
    return df[df["fare"] > 80000]
```

4 alerts/transportation_alert.py

```python
def generate_alert(df):
    alerts = []
    if len(df) > 100:
        alerts.append("🚨 High traffic volume")

    if df["fare"].max() > 90000:
        alerts.append("💰 High fare detected")

    return alerts
```

5 **dashboard/dashboard_transportation.py**

```
import streamlit as st
import time
import sys
import os

# ========================
# FIX MODULE PATH (WAJIB)
# ========================
BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
if BASE_DIR not in sys.path:
    sys.path.append(BASE_DIR)

# ========================
# IMPORT MODULE
# ========================
from analytics import transportation_analytics as ta
from alerts import transportation_alert as alert


# ========================
# CONFIG
# ========================
DATA_PATH = "data/serving/transportation"

st.set_page_config(
    page_title="Smart Transportation Dashboard",
    layout="wide"
)

st.title("🚗 Smart Transportation – Real-Time Analytics")
# ========================
# AUTO REFRESH
# ========================
REFRESH_INTERVAL = 5
placeholder = st.empty()


# ========================
# MAIN LOOP
# ========================
while True:
    with placeholder.container():

        # ========================
        # LOAD DATA
        # ========================
        df = ta.load_data(DATA_PATH)

        if df.empty:
            st.warning("⏳ Waiting for streaming transportation data...")
            time.sleep(REFRESH_INTERVAL)
            continue
```

```
# ========================
# PREPROCESS
# ========================
df = ta.preprocess(df)

# ========================
# METRICS
# ========================
try:
    metrics = ta.compute_metrics(df)
except Exception as e:
    st.error(f"Error computing metrics: {e}")
    time.sleep(REFRESH_INTERVAL)
    continue

col1, col2, col3 = st.columns(3)

col1.metric("Total Trips", metrics["total_trips"])
col2.metric("Total Fare", int(metrics["total_fare"]))
col3.metric("Top Location", metrics["top_location"])

st.divider()

# ========================
# PEAK HOUR
# ========================
try:
    peak_hour = ta.detect_peak_hour(df)
    st.info(f"🚦 Peak traffic hour: {peak_hour}:00")
except Exception:
    st.warning("Tidak dapat menghitung peak hour")
# ========================
# ALERTS
# ========================
try:
    alerts = alert.generate_alert(df)
    if alerts:
        st.subheader("🚨 Traffic Alerts")
        for a in alerts:
            st.error(a)
except Exception as e:
    st.warning(f"Alert error: {e}")

st.divider()

# ========================
# VISUALISASI
# ========================
try:
    col1, col2 = st.columns(2)

    with col1:
        st.subheader("🚗 Fare per Location")
        st.bar_chart(ta.fare_per_location(df))
```

```
            with col2:
                st.subheader("🚚 Vehicle Distribution")
                st.bar_chart(ta.vehicle_distribution(df))

            st.subheader("📈 Mobility Trend")
            st.line_chart(ta.mobility_trend(df))

        except Exception as e:
            st.warning(f"Visualization error: {e}")

        st.divider()

        # ========================
        # ANOMALY
        # ========================
        try:
            st.subheader("⚠️ Abnormal Trips")
            anomaly_df = ta.detect_anomaly(df)

            if not anomaly_df.empty:
                st.dataframe(anomaly_df.tail(20))
            else:
                st.success("No anomalies detected")
        except Exception as e:
            st.warning(f"Anomaly error: {e}")

        st.divider()
        # ========================
        # LIVE DATA
        # ========================
        st.subheader("📋 Live Trip Data")
        st.dataframe(df.tail(50))

    time.sleep(REFRESH_INTERVAL)
```

LANGKAH 3 — Menjalankan Pipeline Transportation

🖥️ TERMINAL 1 — Spark Streaming

Jalankan dari **root project**:

```
spark-submit scripts/transportation/streaming_trip_layer.py
```

👉 Fungsi:

- membaca data dari stream_data/transportation
- menulis ke data/serving/transportation

🖥️ TERMINAL 2 — Trip Generator

```
python scripts/transportation/trip_generator.py
```

👉 Fungsi:

- generate data trip setiap beberapa detik
- masuk ke folder stream_data/transportation

🖥️ **TERMINAL 3 — Dashboard**

```
streamlit run dashboard/dashboard_transportation.py
```

👉 Akses di browser:

`http://localhost:8501`

⚠️ **PENTING (WAJIB DIPATUHI)**

1. **Jalankan dari ROOT PROJECT**

   ```
   cd bigdata-project
   ```

2. **Urutan yang BENAR**

   1️⃣ Spark Streaming
   2️⃣ Generator
   3️⃣ Dashboard

   Kalau 3 terminal ini berjalan:

   `Generator → Streaming → Dashboard`

   👉 Maka sistem sudah:

   🚀 **Real-Time Smart Transportation Analytics Platform**

LANGKAH 4 — Validasi Data

Pastikan folder ini terisi:

`data/serving/transportation/`

LANGKAH 5 — Validasi Dashboard

Dashboard menampilkan:

- *Total Trips*
- *Total Fare*
- *Top Location*
- *Peak Hour*
- *Alerts* 🚨
- *Grafik mobilitas*
- *Tabel data real-time*

LANGKAH 6 — Debugging (Jika Error)

Pastikan:

- __init__.py ada
- menjalankan dari root project
- path data benar
- fungsi analytics lengkap

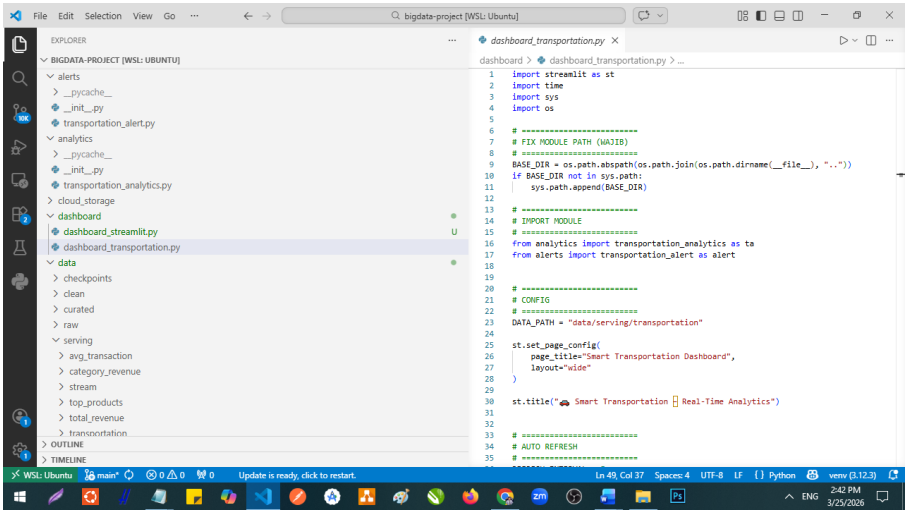
5. CHECKLIST VALIDASI PRAKTIKUM

No	Validasi	Status
1	Generator menghasilkan data	✓
2	Spark streaming berjalan	✓
3	File parquet terbentuk	✓
4	Analytics engine berjalan	✓
5	Dashboard tampil	✓
6	Insight muncul	✓
7	Alert muncul	✓

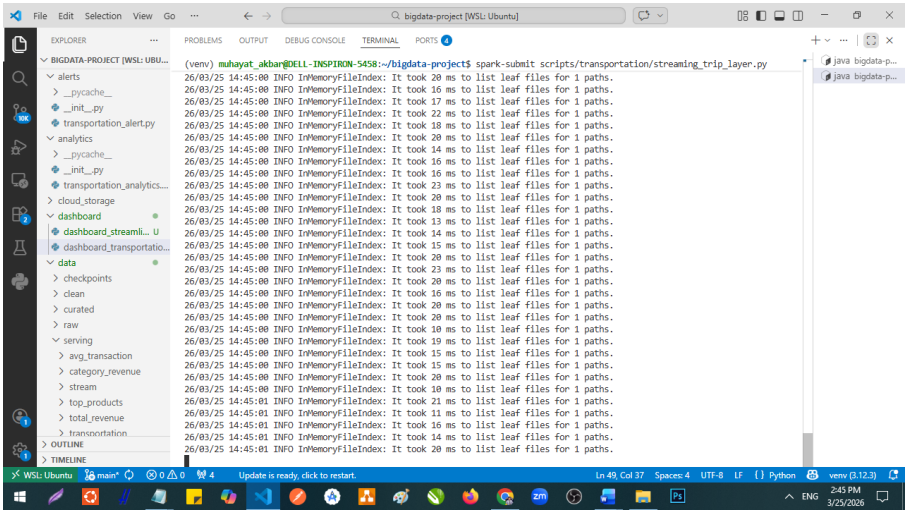
6. OUTPUT WAJIB YANG DIKUMPULKAN

Mahasiswa wajib mengumpulkan:

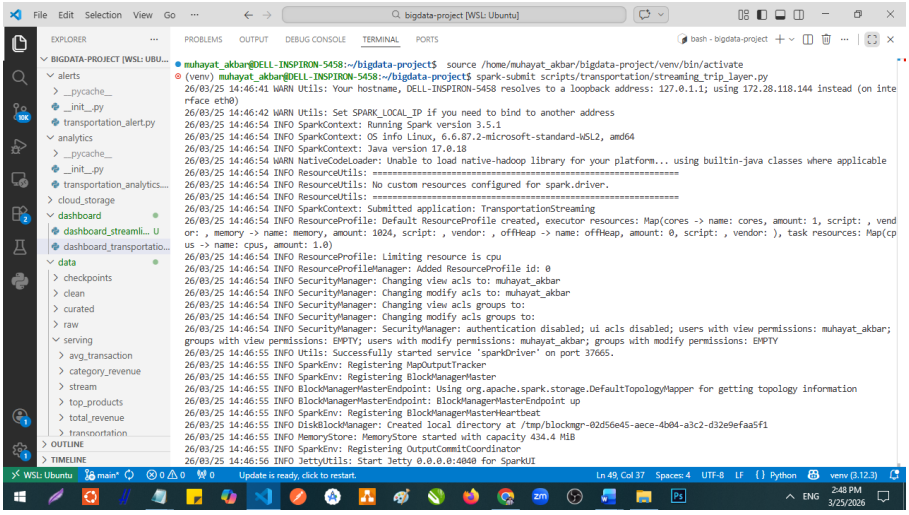
1. Screenshot struktur project



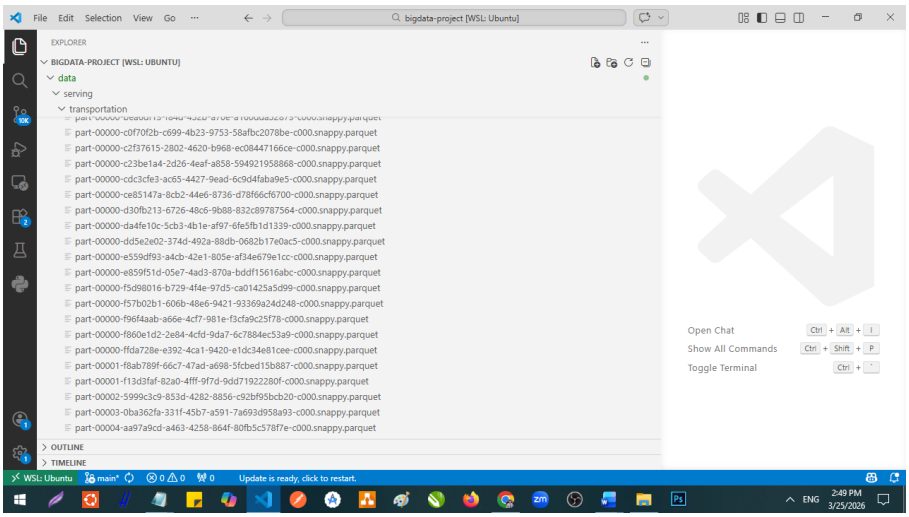
2. Screenshot generator berjalan



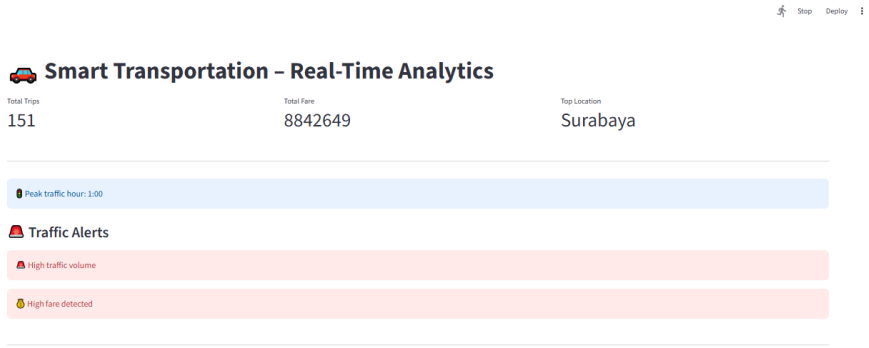
3. Screenshot Spark streaming

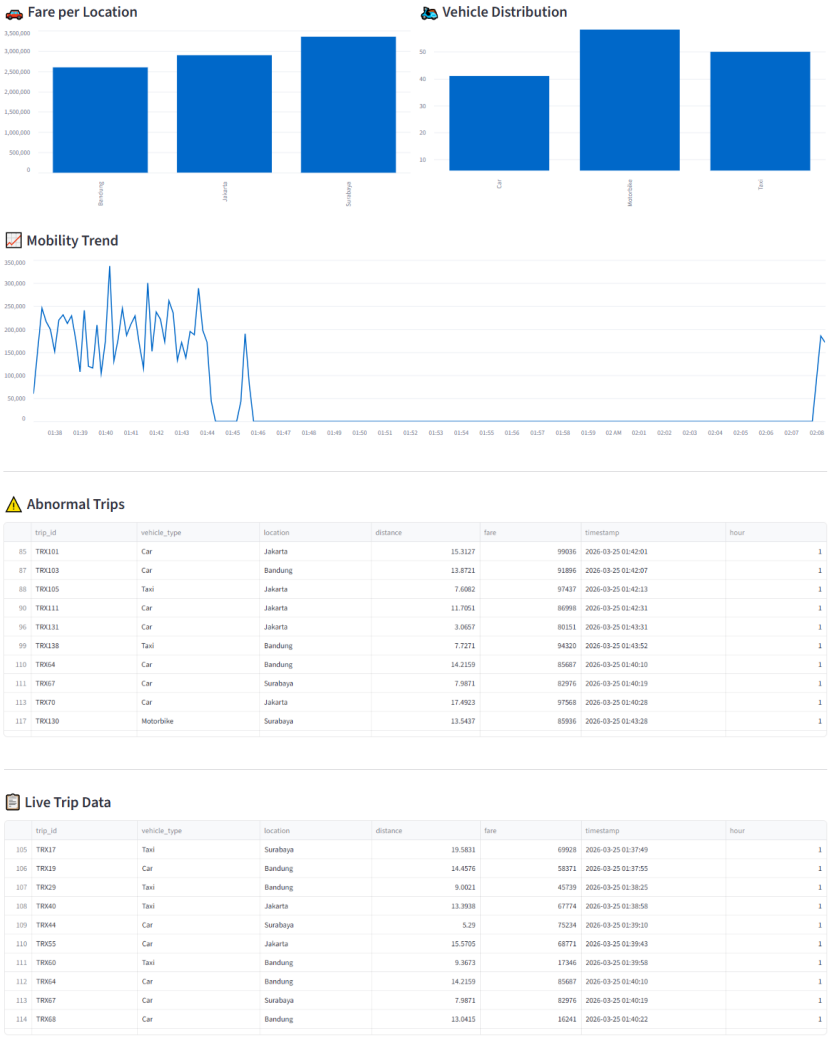


4. Screenshot folder data/serving



5. Screenshot dashboard (insight + alert)





File yang dikumpulkan:

- `transportation_analytics.py`
- `transportation_alert.py`
- `dashboard_transportation.py`

7. RUBRIK PENILAIAN

Aspek	Skor 1	Skor 2	Skor 3	Skor 4
Pipeline	gagal	sebagian	berjalan	stabil
Analytics	tidak ada	basic	cukup	lengkap
Insight	tidak ada	sederhana	cukup	mendalam
Alert System	tidak ada	sederhana	berjalan	optimal
Dashboard	tidak ada	basic	interaktif	advanced

Nilai akhir:
(Nilai = Total Skor / 20) × 100

8. INSIGHT PRAKTIKUM (SMART TRANSPORTATION)

🎯 **Studi Kasus**

Sistem monitoring mobilitas kota (Smart City)

🔍 **Contoh Insight**

1. **Peak Hour**

   "Lonjakan perjalanan terjadi pada jam 17:00 → indikasi jam pulang kerja"

2. **Spatial Insight**

   "Lokasi Jakarta memiliki aktivitas tertinggi → pusat mobilitas"

3. **Behavioral Insight**

   "Kendaraan motor mendominasi → preferensi transportasi cepat"

🚨 **Contoh Alert**

- "Lonjakan volume perjalanan"
- "Tarif perjalanan tinggi"
- "Kemacetan terdeteksi"

🧠 **Kesimpulan Insight**

Sistem mampu:

👉 mendeteksi pola mobilitas
👉 mengidentifikasi kemacetan
👉 memberikan alert real-time

🎯 **PENUTUP**

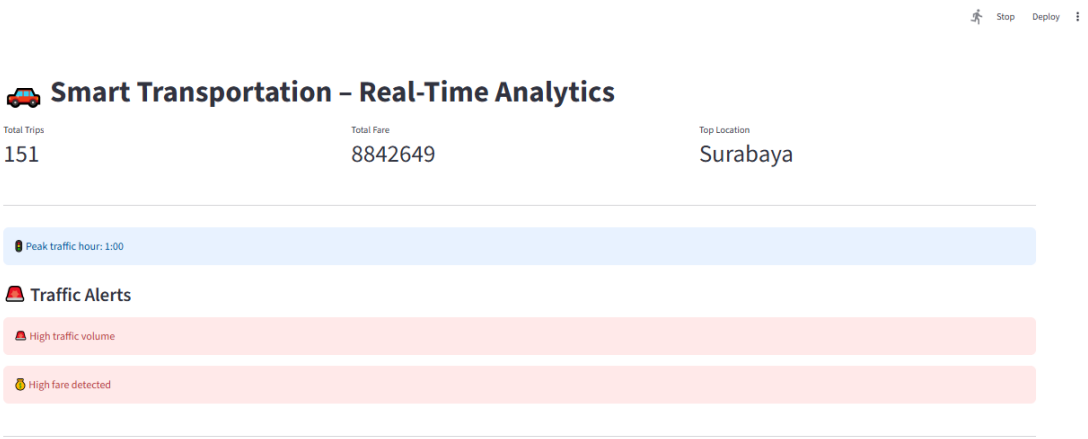
Dengan menyelesaikan praktikum ini, mahasiswa telah memahami:

- *streaming processing*
- *data lake architecture*
- *analytics layer*
- *decision-oriented system*

👉 Ini merupakan fondasi menuju:

- *Machine Learning*
- *Real-Time AI*
- *Smart City Platform*

Insight + Konteks Decision-Oriented System



🔍 1. Overview Metrics (Top Section)

Item:
- Total Trips: **151**
- Revenue: **8842649**
- Top Location: **Surabaya**

Insight:
- Volume trip masih relatif kecil → kemungkinan ini subset data real-time atau periode pendek.
- Revenue tinggi dibanding jumlah trip → indikasi tarif rata-rata tinggi (mungkin long-distance atau premium service).
- Surabaya sebagai top location → demand terpusat, potensi bottleneck atau peluang ekspansi.

Decision-Oriented Use Case:

- 📌 *Strategic allocation*: Fokus armada lebih banyak ke Surabaya.
- 📌 *Pricing optimization*: Evaluasi apakah revenue tinggi karena surge pricing atau jarak.
- 📌 *Market expansion*: Identifikasi kota kedua dengan pertumbuhan cepat.

🚨 2. Traffic Alerts

Item:
- High traffic volume
- Accidents detected

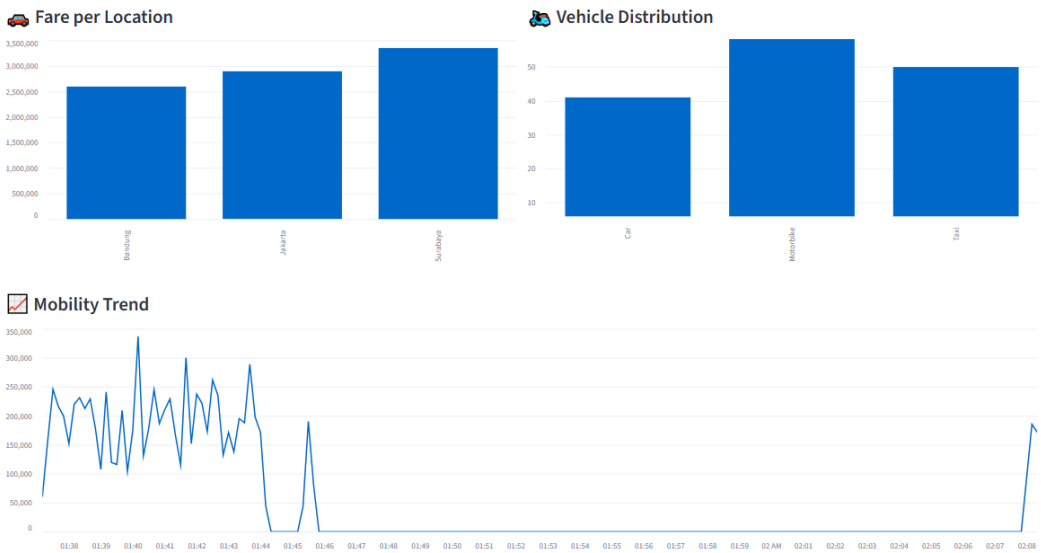
Insight:
- Sistem sudah real-time alert → ini ciri khas **stream processing (Big Data)**.
- Kemacetan + kecelakaan adalah indikator utama gangguan operasional.

Decision-Oriented Use Case:

- 📌 *Real-time rerouting*: Mengalihkan kendaraan dari area macet.
- 📌 *Dynamic ETA adjustment*: Update estimasi waktu pelanggan.
- 📌 *Safety intervention*: Prioritaskan area dengan kecelakaan tinggi.

👉 Ini adalah **core Decision-Oriented System** karena:

- Input: data streaming (sensor/GPS)
- Output: keputusan langsung (routing, alert driver)



💰 3. Fare per Location (Bar Chart)

Insight:

- Perbedaan tarif antar lokasi cukup signifikan.
- Lokasi tertentu menghasilkan revenue lebih tinggi.

Makna Big Data:

- Data ini berasal dari agregasi historis → **batch + real-time hybrid analytics**.

Decision-Oriented Use Case:

- 📌 *Dynamic pricing*: Terapkan surge pricing di lokasi demand tinggi.
- 📌 *Driver incentive*: Dorong driver ke area dengan fare tinggi.
- 📌 *Demand prediction*: Lokasi mahal biasanya high demand.

🚗 4. Vehicle Distribution

Insight:

- Distribusi kendaraan tidak merata.
- Ada potensi mismatch antara supply vs demand.

Masalah yang mungkin:

- Over-supply di area tertentu
- Under-supply di hotspot

Decision-Oriented Use Case:

- 📌 *Fleet balancing*: Redistribusi kendaraan secara real-time.
- 📌 *Idle reduction*: Kurangi kendaraan yang menganggur.
- 📌 *Matching optimization*: Tingkatkan efisiensi pairing driver-passenger.

👉 Ini adalah contoh **optimization problem berbasis Big Data**.

📈 5. Mobility Trend (Time Series)

Insight:
- Fluktuasi tinggi → demand tidak stabil.
- Ada lonjakan dan penurunan ekstrem (spike & drop).

Makna:
- Sistem menghadapi **high variability demand**.
- Perlu predictive analytics.

Decision-Oriented Use Case:
- 📌 *Demand forecasting (ML)*: Prediksi jam sibuk.
- 📌 *Surge pricing otomatis*
- 📌 *Shift scheduling driver*

👉 Ini masuk ke:
Predictive Decision System (lebih advanced dari real-time)

⚠️ Abnormal Trips

	trip_id	vehicle_type	location	distance	fare	timestamp	hour
85	TRX101	Car	Jakarta	15.3127	99036	2026-03-25 01:42:01	1
87	TRX103	Car	Bandung	13.8721	91896	2026-03-25 01:42:07	1
88	TRX105	Taxi	Jakarta	7.6082	97437	2026-03-25 01:42:13	1
90	TRX111	Car	Jakarta	11.7051	86998	2026-03-25 01:42:31	1
96	TRX131	Car	Jakarta	3.0657	80151	2026-03-25 01:43:31	1
99	TRX138	Taxi	Bandung	7.7271	94320	2026-03-25 01:43:52	1
110	TRX64	Car	Bandung	14.2159	85687	2026-03-25 01:40:10	1
111	TRX67	Car	Surabaya	7.9871	82976	2026-03-25 01:40:19	1
113	TRX70	Car	Jakarta	17.4923	97568	2026-03-25 01:40:28	1
117	TRX130	Motorbike	Surabaya	13.5437	85936	2026-03-25 01:43:28	1

⚠️ 6. Abnormal Trips

Isi:
- Trip dengan jarak/timing tidak wajar
- Bisa melibatkan mobil, taxi, dll.

Insight:
- Indikasi:
  - ✓ Fraud
  - ✓ GPS error
  - ✓ Perilaku tidak normal

Decision-Oriented Use Case:
- 📌 *Fraud detection system*
- 📌 *Driver monitoring*
- 📌 *Anomaly alert otomatis*

👉 Ini dalah contoh:
AI-driven anomaly detection (Big Data + ML)

📋 Live Trip Data

	trip_id	vehicle_type	location	distance	fare	timestamp	hour
105	TRX17	Taxi	Surabaya	19.5831	69928	2026-03-25 01:37:49	1
106	TRX19	Car	Bandung	14.4576	58371	2026-03-25 01:37:55	1
107	TRX29	Taxi	Bandung	9.0021	45739	2026-03-25 01:38:25	1
108	TRX40	Taxi	Jakarta	13.3938	67774	2026-03-25 01:38:58	1
109	TRX44	Car	Surabaya	5.29	75234	2026-03-25 01:39:10	1
110	TRX55	Car	Jakarta	15.5705	68771	2026-03-25 01:39:43	1
111	TRX60	Taxi	Bandung	9.3673	17346	2026-03-25 01:39:58	1
112	TRX64	Car	Bandung	14.2159	85687	2026-03-25 01:40:10	1
113	TRX67	Car	Surabaya	7.9871	82976	2026-03-25 01:40:19	1
114	TRX68	Car	Bandung	13.0415	16241	2026-03-25 01:40:22	1

🚕 7. Live Trip Data

Insight:

- Data operasional real-time:
  - ✓ lokasi
  - ✓ jarak
  - ✓ ETA
- Ini adalah data granular (row-level streaming)

Decision-Oriented Use Case:

- 📌 *Real-time dispatch system*
- 📌 *ETA recalculation*
- 📌 *Customer tracking & transparency*

👉 Ini merupakan:
Operational Decision System (ODS)

🎯 Kesimpulan: Letak Decision-Oriented System dalam Dashboard

Dashboard ini sebenarnya mengandung **3 layer Decision System dalam Big Data**:

1. 🔴 Real-Time Decision (Streaming)

- Traffic Alerts
- Live Trip Data

➡️ Keputusan: instan (routing, ETA, dispatch)

2. 🟡 Tactical Decision (Near Real-Time)

- Vehicle Distribution
- Fare per Location

➡️ Keputusan: redistribusi armada, pricing

3. 🟢 Strategic Decision (Historical + Predictive)

- Mobility Trend
- Abnormal Trips
- Overview Metrics

➡️ Keputusan: forecasting, fraud detection, ekspansi bisnis

🧠 Insight Utama untuk Mahasiswa

Dashboard ini adalah contoh nyata implementasi:

- **Lambda/Kappa Architecture (Big Data)**
- Kombinasi:
  - ✓ *Streaming analytics (real-time)*
  - ✓ *Batch analytics (historical)*
  - ✓ *Machine learning (predictive & anomaly)*

👉 Intinya:

"Data tidak hanya ditampilkan, tetapi langsung digunakan untuk mengambil keputusan operasional, taktis, dan strategis."